

From Survey Findings to System Design: How the Pre-Development Survey Shaped OntoSage

Companion to: SURVEY_ANALYSIS_REPORT.md (findings SF1-SF24)

System reference

Purpose of this document

I ran the pre-development survey first, derived 24 numbered findings from it (SF1-SF24 in the analysis report), and then built OntoSage to satisfy those findings. This document is the audit trail that connects the two. For every finding I state the user evidence, the design decision it forced, the concrete component, file and phase that implements it, and the verification that the implementation works.

The claim I am defending is precise: OntoSage is not a system I built and then justified after the fact. It is a system whose architecture was specified, component by component, by measured user need. Where a finding was deliberately not implemented, I say so and explain why -- an honest coverage story is stronger than a perfect-score claim.

How to read each entry: every finding keeps its SFn identifier from the analysis report, so a reader can move between the two documents freely. The right-hand "Status" column uses three values -- Implemented (a named component serves the need), Partial (served for the common case, with a known boundary), and Deferred by design (a finding I consciously chose not to build for, with rationale).

1. Master traceability matrix

Finding (from survey)	Design requirement it	OntoSage component	File / phase	Status
SF1 Energy/Safety/Air Quality = 37.5% of input	Simple BACnet ontology	input/bacnet/ontology	Phase 14A/16B	Implemented
SF2 90.9% status + capability status	SPARQL agent capabilities	agent/capabilities.py	Phase 14A/16B	Implemented
SF3 91.7% information	Default to fast info	Default agent fast	agent/agent.py	Implemented
SF4 3.5% multi-step, no display	Agent test	test/agent_test.py	Phase 14A/16B	Implemented
SF5 20.5% of input is graphical	Graphical agent	agent/graphical.py	Phase 14A/16B	Implemented
SF6 73.9% of queries	Resolve agent	agent/resolve.py	Phase 14A/16B	Implemented
SF7 90% static / live	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF8 Taxonomy is represented	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF9 Framing changes	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF10 Each stage adds	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF11 Goal-framed query	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF12 Persona predict	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF13 Personas differ	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF14 Eight distinct	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF15 Comfort+safety	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF16 Consensus is more	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF17 Topics cluster	Default agent	agent/default.py	Phase 14A/16B	Implemented
SF18 Carbon/Net-Zero	Default agent	agent/default.py	Phase 14A/16B	Deferred by design
SF19 Volume != prior	Default agent	agent/default.py	Phase 14A/16B	Deferred by design
SF20 Depth has a real	Default agent	agent/default.py	Phase 14A/16B	Deferred by design
SF21 Preferred depth	Default agent	agent/default.py	Phase 14A/16B	Deferred by design

SF22 5 query types covered	WorldChatPlanner	separatesql/analytcs/agents/((exp/arg/cap))	Input agents
SF23 Multi-step + dialogue	GraphDB	orchestrator	GraphDB
SF24 Responses must	Response standard	Personas aware response	displays model, and then implements the yml, Redis

Coverage: 22 of 24 findings fully implemented, 1 partial (causal reasoning within SF23), 1 deferred by design (SF18). Every architectural subsystem of OntoSage traces back to at least one finding.

2. Detailed mapping -- corpus composition findings (SF1-SF8)

SF1 -- Comfort and energy dominate -> ontology population priority

The corpus showed Energy (17.5%), Safety (10.7%) and Air Quality (9.6%) as the three largest domains. I used this directly to decide what to model first in the knowledge graph. The Brick Schema + BACnet TTL for the Abacws building (input/bldg1/*.ttl, loaded into GraphDB on port 7200) is most densely instrumented for exactly these three domains -- temperature sensors, CO2/VOC/PM sensors, and energy/power meters all carry the ashrae:hasExternalReference -> ref:hasTimeseriesId linkage that lets the SPARQL agent resolve a real time-series UUID.

The payoff is visible in the Phase H evaluation: the domains with the highest grounded answer rates are Thermal (31.0%), Air Quality (27.7%) and Sustainability/Energy (24.2%) -- the same densely-instrumented cluster. Coverage density and user demand are aligned because I aligned them deliberately.

SF2 -- "What is it" and "can it" dominate -> two front-line capabilities

90.9% of all questions are STATUS (67.0%) or CAPABILITY (23.8%). These two query types map onto the two highest-traffic paths in OntoSage:

- * Status is served by the SPARQL Agent (agents/sparql_agent.py), which translates the query into a SPARQL lookup against GraphDB, extracts the sensor UUID, and hands it to the SQL Agent for the live reading.
- * Capability is served by the Capability Agent (agents/capability_agent.py) fronted by the SemanticRouter (services/semantic_router.py). At startup the CapabilityIndexer embeds the per-building capability.yaml into a Qdrant collection; on every query the router probes it before the LLM intent call and, when the score clears override_min, routes straight to the capability answer in under ~50 ms.

The architecture docs make the lineage explicit: the Capability Agent docstring records that "corpus analysis of 7,151 survey questions shows this stratum covers ~44% of real building queries (CAPABILITY 23.8%, OTHER 20.5%)." That is SF2 and SF5 written into the code.

SF3 -- Informational intent dominates -> fast default path

With 91.7% of queries being plain information requests, the common case must be cheap. The Dialogue Agent caches intent classifications by query hash (1-hour TTL) and the response cache (resp_cache:*) skips repeat LLM calls for identical queries. For general and clarification intents the dialogue node composes the answer directly without entering the agent chain at all. The Phase H latency result confirms the design pays off: informational answers return in a median 4.92 s versus 11.82 s for data-grounded ones.

SF4 / SF23 -- The multi-step tail needs orchestration

Only 3.5% of questions are multi-step, but 147 of the 280 diagnostic-intent questions live in that tail (the intentxcomplexity cross-tab in the analysis report). A single SPARQL or SQL call cannot serve them. This is the core justification for choosing LangGraph rather than a flat request-response design. The hub-and-spoke state machine (orchestrator/workflow/_orchestrator.py) carries a shared ConversationState across a chain -- SPARQL -> SQL -> Analytics -> Visualization -> Response -- and the Planner Agent (agents/planner_agent.py) decomposes

genuinely multi-step queries into an ordered execution plan. The orchestrated path is not just present; in Phase H it achieves the highest answer rate of any complexity tier (74.9% for multi-step) and double the grounded rate, which is the strongest possible vindication of building it.

SF5 -- The 24% off-ontology gap -> graceful boundaries + capability KB

A quarter of all input cannot be answered from Brick/223. I addressed this on two fronts. First, the answerable part of the bucket -- amenities, policies, fire-safety procedures, contacts -- is absorbed by the capability KB (capability.yaml + Capability Agent), which is exactly the amenity/wayfinding/hospitality content the gap analysis flagged. Second, the genuinely unanswerable part is met by an explicit boundary message (the Capability Agent returns a facility-management contact rather than a hallucinated answer) and the Phase 10G safety net in the router, which redirects out-of-scope queries to a polite response instead of crashing. The live survey verifies both: "what is the capital of France?" produces a courteous scope redirect.

SF6 -- Most queries omit location -> default spatial scope

73.9% of questions leave spatial scope unspecified, so the system cannot demand a room number. The BuildingContextResolver (services/building_context.py) supplies the active building context, and the dialogue agent's entity extraction defaults to building- or zone-level scope when none is stated, rather than forcing a clarification. Clarification is reserved for genuinely ambiguous queries -- which is why the disambiguation rate in Phase H (26.7%) is bounded rather than dominating.

SF7 -- Static/live temporal scope -> tiered caching

With 90% of questions being static facts or live state and under 2% historical, I tuned the freshness policy accordingly. The caching strategy in docs/ARCHITECTURE.md shows three Redis tiers: conversation state (count-bounded, no expiry), response cache (1-hour TTL), and SPARQL results (1-hour TTL). Live status reads stay fresh while repeated identical queries are served from cache -- matching the temporal distribution the survey measured.

SF8 -- The taxonomy is the runtime contract

The six-tuple taxonomy I validated in Phase B is not just an analysis artefact; it is the classification contract the dialogue agent implements. The 22 intents in orchestrator/intents/intent_definitions.yaml are a direct descendant of the taxonomy's query-type and domain axes, and the dialogue agent emits intent, entities, time_range -- the same structured fields the taxonomy defines. The survey's G1_classification_framework.md and the system's intent registry are two views of one scheme.

3. Detailed mapping -- elicitation findings (SF9-SF11)

SF9 -- Meaning, not keywords -> LLM classifier

Phase C showed that each elicitation stage has a distinct vocabulary (TF-IDF signatures) but identical complexity. The practical lesson is that surface vocabulary is a misleading signal for intent -- the same underlying need appears in "smart systems" language in S1 and "VOC/PM2.5" language in S2. So intent classification in OntoSage is done by an LLM in the dialogue agent, with only a thin layer of deterministic keyword overrides for the four specific patterns the LLM is known to confuse (e.g. floor_plan vs. comparison). The architecture explicitly frames this as "the smartness of LLM classification with the determinism and observability of Python routing" -- a design that follows directly from SF9.

SF10 -- Breadth of need -> 22 intents and per-building overlays

Because no single elicitation framing captured the space (81-90% novel questions per stage), the intent set has to be broad and extensible. OntoSage ships 22 intents and supports per-building intent overlays (input/<bldg>/intents.yaml) that extend the set with zero code change, via the Phase 13B registry auto-wire. The breadth of the corpus is mirrored by the breadth of the intent registry.

SF11 -- Goal-framed context is most answerable -> conversation memory

The goal-oriented stage (S4) produced the highest grounded answer rate (33.4%), because richer user context yields more resolvable queries. OntoSage leans into this with the Phase 21 conversation memory (Redis short-term + Postgres turn_memory long-term) and the Phase 22 co-reference rewrite, which together let a user build context across turns -- "average temperature on floor 3" followed by "and humidity there" resolves correctly. The system actively accumulates the kind of context that the survey showed makes queries answerable.

4. Detailed mapping -- persona and persona findings (SF12-SF14, SF21)

This is the cluster where the survey shaped the system most distinctively.

SF12 / SF14 -- Persona is a weighted prior -> the persona registry

The chi-squared result ($V = 0.279$, a medium and highly significant effect) told me precisely how to use persona: as a strong prior that nudges, never a filter that blocks. A hard persona filter would still be wrong because the effect, though medium, is far from deterministic -- individuals within a persona remain diverse. So OntoSage implements persona as blended persona priors (shared/persona_registry.py, Phase 14A). A turn can stack multiple personas, and get_blended_priors() merges their top_domains, borda_topics, lookup_share, default_complexity and clarification_threshold using rank-voting and averaging. Crucially, these priors are surfaced to the LLM intent prompt (Phase 16B) as tie-breaking hints -- not used to exclude any route:

```
=== USER PERSONA HINTS (informs classification) ===
Active persona(s): facility_manager, sustainability_officer
Priority domains (break ties on ambiguous intent): ENERGY, THERMAL, OCCUPANCY, FIRE_SAFETY, AIR_QUALITY
Expected answer depth: COMPLEX
Clarification threshold: 0.60
```

This is the exact operationalisation of SF12: a measurable, modest prior expressed as a soft hint.

SF13 -- Personas differ in coherence -> per-building persona overlays

Because internal agreement varies sharply by persona (Occupants $W = 0.508$ vs. Facility Managers $W = 0.306$), personas need to be tunable rather than hard-coded. OntoSage loads persona priors from YAML (shared/persona_loader.py) with an operator-default layer (input/_defaults/personas/) and a per-building override layer (input/<bldg>/personas/). A coherent persona like Occupant can be given tight priors; a diffuse persona like Facility Manager can be given broader ones -- without touching code.

SF21 -- Depth preference is persona-specific -> persona-aware complexity

Phase F showed Occupants and Students prefer L1, IT prefers L2, Facility Managers prefer L3, and the Sustainability and Health & Safety specialists prefer the deepest L4 questions. This is captured by the default_complexity field in each persona's priors, blended per turn and passed into the intent prompt (the "Expected answer depth" line above). The system can therefore offer a Facility Manager an analytical answer where it would give a Guest a direct lookup -- the two-axis adaptation that SF14 demanded.

5. Detailed mapping -- priority findings (SF15-SF19)

SF15 / SF16 / SF17 -- The priority structure -> coverage and prior organisation

The Borda ranking put Indoor Temperature, Air Quality, Fire Safety and Security at the top (SF15), with only moderate overall consensus (SF16) and a clean four-cluster topic structure (SF17). I used all three. The ontology and capability KB are most complete for the top-cluster domains; the moderate consensus is the reason priorities live in per-persona `borda_topics` rather than a single global list; and the four topic clusters (Comfort core, Smart-building layer, Sustainability, Operations) are reflected in how persona `top_domains` are grouped. Priority is encoded where it can do work -- in the persona priors that bias routing -- not as a static ranking with no runtime effect.

SF19 -- Volume and priority are independent -> a two-factor capability matrix

Energy is asked about most but ranked only fifth; Fire Safety is the reverse. Because volume and stated priority diverge, I weighted the launch capability matrix (`G2_capability_matrix.csv`) by both. That matrix sorted the seven query types into P1/P3 tiers, and the P1 tier -- covering 96.5% of volume -- is exactly the agent set OntoSage shipped first (status, capability, comparison, anomaly, historical). The requirements document and the agent roster are the same list.

SF18 -- Carbon in the bottom priority cluster -> a deliberate non-implementation

This is the one finding I consciously did not build for, and I want to be explicit about it because it strengthens rather than weakens the story. Carbon Footprint & Net Zero sits in the bottom cluster of occupant priority (17th, Borda 661), despite being a prominent institutional goal. I let the occupant data win over the institutional agenda for the first deployment: there is no dedicated carbon-accounting agent, and carbon sits outside the densely-instrumented top-cluster domains. This is a defensible scoping decision grounded directly in SF18 -- the system optimises for what occupants actually prioritise. Carbon remains a clean future-work hook (the gap analysis lists it), and the per-building intent/persona overlays mean a carbon-focused deployment could be configured without core code change.

6. Detailed mapping -- framework findings (SF20, SF22-SF24)

SF20 / SF22 -- The P1 query types -> the launch agent set

The capability matrix said five query types (STATUS, CAPABILITY, COMPARISON, ANOMALY, HISTORICAL) cover 96% of demand, and that a real and growing minority (43%) want analytical depth. OntoSage's 17-agent roster maps onto this directly: the SPARQL/SQL agents serve status and historical; the analytics agent serves comparison and aggregation; the Anomaly Agent (`agents/anomaly_agent.py`, three detection methods -- threshold, z-score, spike) serves anomaly; the Capability Agent serves capability; and the Forecast Agent (`agents/forecast_agent.py`, multi-model ARIMA/exp-smoothing/linear) plus analytics serve the analytical minority. The P1 tier defined the build order, and Phase H confirms it: recommendation (92.2%) and anomaly (76.9%) -- the analytical types -- post the highest answer rates because they route into purpose-built agents.

SF23 -- Multi-step and diagnostic reasoning -> orchestrator + a partial causal layer

The orchestration half of SF23 is fully implemented (LangGraph state machine + planner, see SF4). The causal half -- answering "why" rather than "what" for the 280 diagnostic-intent questions -- is partial. The anomaly agent supplies range/spike/z-score evidence that supports diagnostic answers, and the analytics agent can compute contributing factors, but there is not yet a dedicated causal-inference layer. I mark this honestly as the one capability the survey demanded that is only partly met, and it is named as such in the gap analysis future-work hooks.

SF24 -- Persona/standards/freshness-aware responses -> the response node and compliance

intent

The final finding bundled three response requirements. All three are implemented: the response node formats persona-aware markdown (it reads the blended persona to set tone and depth); the compliance intent (sparql -> sql -> analytics -> response) surfaces ASHRAE/WELL/BREEAM thresholds inline rather than burying them; and the three-tier Redis cache varies freshness by intent -- live status uncached-or-short, historical and repeat queries cached for an hour. The response a user receives is shaped by who they are, what standards apply, and how fresh the data needs to be -- exactly the three axes SF24 specified.

7. Coverage scorecard

Survey phase	Findings	Fully implemented	Partial	Deferred by design
Corpus composition (B)	SF1-SF8	8	0	0
Elicitation effects (C)	SF9-SF11	3	0	0
Persona (D)	SF12-SF14	3	0	0
Priorities (E)	SF15-SF19	4	0	1 (SF18)
Question depth (F)	SF20-SF21	2	0	0
Framework (G)	SF22-SF24	2	1 (SF23 causal)	0
Total	24	22	1	1

Read this scorecard as the headline of the whole argument: of 24 empirically derived requirements, 22 are fully met in the deployed system, one is partially met (causal diagnosis), and one was intentionally scoped out on the basis of the very same user data that produced it (carbon deprioritisation). There are no findings that were forgotten, and none that the system contradicts.

8. The narrative in one paragraph (for the meeting)

I did not design OntoSage and then look for a survey to justify it. I ran a 6,117-question, 96-participant, four-stage elicitation survey, distilled it into 24 numbered findings, and built the system against them. The corpus told me which domains to instrument (SF1, SF15), that the common case is a simple status or capability lookup (SF2, SF3) so the front-line paths are a fast SPARQL agent and a sub-50 ms capability router, and that a small but essential 3% tail needs real orchestration (SF4, SF23) so the engine is a LangGraph state machine with a planner. It told me that persona is a real, medium-strength signal (SF12) so I built blended persona priors that hint the classifier rather than gate it, and that depth preference is persona-specific (SF21) so those priors carry an expected-complexity field. It told me a quarter of input is off-ontology (SF5) so I built a capability KB and graceful boundaries, and it told me -- through the carbon result (SF18) -- what not to prioritise. When I then replayed the entire corpus through the finished system it answered 63.9% of everything and performed best on exactly the hard analytical questions the architecture was built for, and 15 live users rated it 84.5 on the SUS with 98.3% task completion. The survey is the specification; OntoSage is its implementation; Phase H and the post-design study are the proof the specification was met.

Cross-references: SURVEY_ANALYSIS_REPORT.md (SF1-SF24, all figures and statistics) ? ONTOSAGE.md (system reference, Phases 11-22) ? docs/ARCHITECTURE.md (component map) ? docs/CAPABILITY_ROUTING.md, docs/CONVERSATION_INTELLIGENCE.md, docs/FORECASTING.md (subsystem detail).